

**HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND
EDUCATION
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING
DEPARTMENT OF COMPUTER AND COMMUNICATION
ENGINEERING**



**FINAL COURSE PROJECT
SMART WATER TANK: LEVEL AND
FLOW MONITORING WITH
AUTOMATIC PUMP CONTROL
MAJOR IN COMPUTER ENGINEERING**

Students: [FULL NAME OF MEMBER 1]
Student ID: [ID]
[FULL NAME OF MEMBER 2]
Student ID: [ID]
[FULL NAME OF MEMBER 3]
Student ID: [ID]
[FULL NAME OF MEMBER 4]
Student ID: [ID]
[FULL NAME OF MEMBER 5]
Student ID: [ID]

Supervisor: Dr. HUYNH THE THIEN

HO CHI MINH CITY, 10/2026

**HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND
EDUCATION
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING
DEPARTMENT OF COMPUTER AND COMMUNICATION
ENGINEERING**



**FINAL COURSE PROJECT
SMART WATER TANK: LEVEL AND
FLOW MONITORING WITH
AUTOMATIC PUMP CONTROL
MAJOR IN COMPUTER ENGINEERING**

Students: [FULL NAME OF MEMBER 1]
Student ID: [ID]
[FULL NAME OF MEMBER 2]
Student ID: [ID]
[FULL NAME OF MEMBER 3]
Student ID: [ID]
[FULL NAME OF MEMBER 4]
Student ID: [ID]
[FULL NAME OF MEMBER 5]
Student ID: [ID]

Supervisor: Dr. HUYNH THE THIEN

HO CHI MINH CITY, 10/2026

ACKNOWLEDGMENTS

Throughout this final project for the course IoT Foundations and Applications, our group received careful guidance and valuable technical feedback from Dr. Huynh The Thien. His lectures on layered architecture, application layer messaging and time series data infrastructure provided the theoretical basis for most of the design decisions in the system we built.

We also thank the Department of Computer and Communication Engineering for granting us laboratory access, which allowed us to complete the hardware prototype and carry out the measurements reported here.

We would like to express our sincere and deepest gratitude.

Project group
(Signature and full name)

DECLARATION OF AUTHORSHIP

We declare that the entire content presented in this report is the result of our own research and implementation. The firmware, the backend service and the monitoring interface were written by us. The experimental data were collected directly on the physical prototype that we assembled. Every item taken from the course lectures, technical standards and scientific literature is cited at the point of use and listed in the references.

Project group
(Signature and full name)

ABSTRACT

This project builds a closed loop smart water tank for Project 08: it measures water level and flow, drives a pump automatically, accumulates consumed volume, detects and classifies faults, and can be supervised from anywhere on the Internet. The system spans the layered model of the Internet of Things: an ESP32 with an ultrasonic level sensor, two turbine flow sensors, a Hall current sensor and two float switches; MQTT over Wi-Fi to a Mosquitto broker; a FastAPI service with a SQLite time series store; and a web dashboard published through an HTTPS tunnel, where anyone may watch and only a password authenticated administrator may control.

The central design decision is to place the entire control law and every safety rule inside the edge microcontroller. Level regulation therefore keeps working while the network is down, and a ring buffer replays telemetry when it returns. The controller is a seven state machine with a 30 to 70 percent hysteresis band and minimum on and off times; eleven fault rules, a single guard function through which every pump start must pass, and independent overflow layers ensure that manual commands cannot bypass safety. Level readings pass a robust filter chain (median window, interquartile spread gate, physical rate checks and an alpha-beta filter), and a pump health classifier separates hydraulic from electrical pump faults using current and level evidence. A rolling baseline over 48 half hour slots detects abnormal consumption on the backend.

On the bench, every automatic fill stopped between 70.1 and 70.8 percent with at most 1.85 percent overshoot, the pump started 12.9 times per hour with no relay chatter, no false alarm was raised in 84 minutes, pumped volume was estimated with a mean error of -5.2 percent against a geometric reference, and commands were confirmed with a median round trip of 42 milliseconds. The report states openly where the prototype falls short, chiefly that conducted interference from the pump makes the turbine flow sensors unusable in this installation, so flow is derived from the level.

Contents

1	INTRODUCTION	1
1.1	Motivation and problem statement	1
1.2	Scope and working method	1
2	THEORETICAL BACKGROUND AND SYSTEM REQUIREMENTS	3
2.1	Requirements of the assignment	3
2.2	Layered architecture and computation placement	5
2.3	Sensing, digitisation and actuation	5
2.4	Application layer messaging	6
2.5	Data infrastructure and the feedback sync principle	6
2.6	On off control with hysteresis	7
3	ARCHITECTURE AND HARDWARE DESIGN	8
3.1	Overall architecture	8
3.2	Function placement and edge autonomy	10
3.3	Sensor selection and signal conditioning	10
3.4	Power stage, supply and mechanical build	12
4	PROTOCOL AND COMMUNICATION	13
4.1	Design principles and topic tree	13
4.2	Message structure and fault codes	14
4.3	Command and acknowledgement mechanism	15
4.4	Connection lifecycle and availability monitoring	15
5	BACKEND, DATA MANAGEMENT AND INTERFACE	17
5.1	Backend architecture	17
5.2	Data schema and API	18
5.3	Monitoring interface	20
5.4	Device simulator	21
6	CONTROL AND SYSTEM INTELLIGENCE	22
6.1	Overview	22
6.2	Computed quantities	22
6.3	State machine and hysteresis	24

6.4	Fault detection rules	25
6.5	Manual mode and safety interlocks	26
6.6	Leak and anomalous consumption detection	26
6.7	Pump health classification	27
7	SECURITY AND RELIABILITY	28
7.1	Threat model and mitigations	28
7.2	Authentication, authorisation and encryption	29
7.3	Reliability during a network outage	29
8	EXPERIMENTS, RESULTS AND CONCLUSION	31
8.1	Principles and reference instruments	31
8.2	Sensor calibration	32
8.3	Control response and switching	34
8.4	Fault detection	35
8.5	Latency and behaviour during an outage	36
8.6	Reducing transport latency at the radio	38
8.7	Limitations	39
8.8	Faults found on the bench	40
8.8.1	An inverted relay made the firmware believe the pump was off	40
8.8.2	An automatic direction level shifter consumed a third of the echoes	41
8.8.3	Fifty hertz on a floating signal line reads as plausible flow	41
8.8.4	A silently truncated message stopped the database without stopping anything else	41
8.8.5	A command that worked but was never confirmed	42
8.8.6	A robust filter gated by a fragile statistic	42
8.8.7	The level sensor is excellent, and unusable, in the same installation	43
8.9	Conclusion	43
8.10	Member contributions	44

List of Tables

2.1.1 Requirements of Project 08 and where they are addressed.	4
3.3.1 Main bill of materials.	10
4.1.1 MQTT topic tree and the reasoning behind each setting.	13
5.2.1 Main API endpoints.	18
6.3.1 Controller state set.	24
6.4.1 Fault detection rules and trigger conditions.	25
7.1.1 Threat scenarios and mitigations.	28
7.3.1 Independent layers of overflow protection.	30
8.1.1 Experiment plan and status.	32
8.2.1 E1 level calibration against the scale.	32
8.2.2 YF-S401 pulse rates on the bench.	33
8.2.3 E2 and E3: complete automatic fill cycles, 23 September.	34
8.4.1 E5 fault injection.	36
8.5.1 Latency and outage results.	37
8.6.1 Command round trip before and after disabling Wi-Fi power saving.	38
8.10.1 Distribution of work among group members.	44

List of Figures

3.1.1 Architecture of the smart water tank system. The closed control loop lives entirely inside the edge node.	9
5.2.1 Daily volume pumped in and drawn out, as served by the API.	19

5.3.1 Monitoring dashboard during an automatic fill: process mimic with the tank at 60.5 per-
cent, numeric tiles with the pump health label, and the level history. For this screenshot,
recorded bench data from 23 September were replayed into an isolated copy of the backend,
so the live database was not modified. 20

8.3.1 Level during the analysed session. Blue bands: pump running. Hatched: manual mode,
used for command tests and for draining the tank below the band on purpose. 34

8.5.1 Distribution of command round trip times in the analysed session. 37

List of Abbreviations

The following abbreviations are used throughout this report.

Abbreviation	Definition
IoT	Internet of Things
MQTT	Message Queuing Telemetry Transport
QoS	Quality of Service
LWT	Last Will and Testament
ACL	Access Control List
TLS	Transport Layer Security
REST	Representational State Transfer
API	Application Programming Interface
JSON	JavaScript Object Notation
IQR	Interquartile Range
mDNS	Multicast Domain Name System
RTT	Round Trip Time
HTTPS	Hypertext Transfer Protocol Secure
TSDB	Time Series Database
ADC	Analog to Digital Converter
SAR	Successive Approximation Register
PWM	Pulse Width Modulation
GPIO	General Purpose Input Output
MCU	Microcontroller Unit
FSM	Finite State Machine
ISR	Interrupt Service Routine
NVS	Non-Volatile Storage
EMF	Electromotive Force
MAE	Mean Absolute Error
RMSE	Root Mean Square Error
EWMA	Exponentially Weighted Moving Average
CUSUM	Cumulative Sum

Chapter 1

INTRODUCTION

1.1 Motivation and problem statement

Household water management in Vietnam still relies largely on a mechanical float placed inside the storage tank. The device is cheap and durable, but it solves exactly one problem: cutting the pump when the tank is full. The owner has no idea how much water was consumed, cannot notice a pipe leak until the bill rises unexpectedly, and has no way of knowing that the pump is running dry until it has already failed.

The problem carries every characteristic of a cyber physical system, meaning a system in which computation, communication and a physical process are tied together through feedback loops [1]. A physical quantity is measured, digitised, transported to a data infrastructure, turned into a decision, and finally acted back upon the same physical process through an actuator.

From an engineering point of view the tank is a nonlinear plant carrying a hard safety constraint. Discharge flow depends on the square root of the water column height, so the plant response changes with the current level. At the same time there is one constraint that may never be violated under any circumstance, including a manual command issued by an operator: the tank must not overflow. Having to guarantee control quality and an absolute safety constraint at the same time is what lifts this project above a simple exercise in wiring a sensor to a display.

The system must therefore address four groups of problems: measuring level and flow with an error budget established experimentally rather than copied from a datasheet; holding the level inside a desired band without letting the pump chatter around the switching threshold; detecting abnormal hydraulic operating modes; and providing remote supervision through a time series store and a real time interface. Overarching all of them is a single requirement that shapes the whole architecture: essential level control must keep working when the network link is lost.

1.2 Scope and working method

The system is built at laboratory prototype scale with one sensing node, a tank with a 1.4 litre working volume and a low voltage direct current pump. The entire electrical side operates at safe extra

low voltage. Distributed multi node deployment, machine learning based consumption forecasting and integration with a commercial cloud platform are outside the scope of this work.

The group organised the work along the V model introduced in the project management part of the course [1], in which every decomposition step on the descending branch has a matching verification step on the ascending branch: requirement analysis pairs with full system acceptance, architectural design pairs with integration testing, protocol specification pairs with protocol level testing, and circuit design pairs with unit testing of each individual sensor.

One methodological decision deserves emphasis. The group specified the complete MQTT topic tree, the message payloads, the set of API endpoints and the database schema before writing a single line of code. That specification acted as an internal contract, which allowed the firmware branch, the backend branch and the interface branch to advance in parallel instead of waiting on one another. While components had not yet arrived, a simulator publishing messages that conform to the agreed specification let the other two branches be developed and tested in full.

Chapter 2

THEORETICAL BACKGROUND AND SYSTEM REQUIREMENTS

2.1 Requirements of the assignment

The assignment, Project 08, asks for a smart water tank that monitors consumption and controls its pump automatically. Table 2.1.1 restates each requirement in the words of the brief and points to where the report answers it. The third column is the group's own assessment of how completely the requirement is met, because a requirement that is only partly met is better stated than hidden.

Table 2.1.1: Requirements of Project 08 and where they are addressed.

Requirement	Where	Status
ESP32 class controller with tank level and flow sensors; pressure and pump current optional	Sections 3.3, 6.2	Met; both flow sensors fitted, flow derived from level (Section 3.3)
Low voltage pump through a relay or MOS-FET driver with inductive protection	Section 3.4	Met
Prevent overflow and behave safely on failed or implausible sensors	Sections 6.4, 7.3	Met, six independent layers
Instantaneous flow and accumulated volume	Section 6.2	Met, volume error -5%
Publish telemetry, actuator state, faults and availability over MQTT	Chapter 4	Met
Time series storage, REST API, dashboard with level, flow, daily volume, pump state, alarms and freshness	Chapter 5	Met
Automatic control with hysteresis or a state machine and at least one fault rule, such as pump on with near zero flow	Sections 6.3, 6.4	Met, eleven rules
Manual mode cannot bypass safety; commands acknowledged with confirmed state	Sections 6.5, 4.3	Met
Local level control keeps working during a network outage	Sections 3.2, 7.3, 8.5	Met
Advanced: leak detection from a rolling baseline, or pump current and flow consistency to classify pump faults	Sections 6.6, 6.7	Both built; baseline still converging
Calibrate level and flow against a physical reference	Section 8.2	Flow met; level pending final run
Control response, switching frequency, fault detection delay, volume estimation error	Sections 8.2 to 8.4	Met
Sensor or no flow fault test, network loss and recovery, telemetry and control latency	Sections 8.4, 8.5	Met; no flow delay pending final run

2.2 Layered architecture and computation placement

An Internet of Things deployment is commonly described by a five layer model made of perception, transport, processing, application and business layers, allowing each stage of signal conversion, transport and storage to be optimised independently [1]. Recommendation ITU-T Y.2060 adds two capability groups that cut across the stack, management and security [2]. Treating security as cross cutting rather than as a separate layer has a direct consequence for our design: it cannot be handled at a single point but must appear in broker authentication, per topic access control, transport encryption and API authorisation. Data flow is likewise split into a southbound path carrying commands down to the hardware and a northbound path carrying telemetry and alerts up to the server [1].

Deciding where a computation belongs is driven by quantitative constraints rather than taste. Edge computation offers deterministic latency and very low bandwidth cost but only short term storage, while cloud computation offers durable storage and analytic capacity at the price of latency from hundreds of milliseconds to several seconds [1]. The resulting rule is that any operation requiring an immediate safe stop must be computed at the edge, whereas trend analysis is routed upward. Cutting the pump on a dry run condition or when the level passes the safety threshold clearly belongs to the first category, which is the direct basis for the decision in Section 3.2.

2.3 Sensing, digitisation and actuation

Faithful reconstruction of a continuous analog signal requires a sampling rate greater than twice its highest frequency component [3]. With the pump actually fitted and the tank base measuring 10 by 10 centimetres, the group measured the fill rate directly rather than deriving it from the datasheet. The level rose 1.2 centimetres in 20 seconds, that is 0.06 centimetres per second, equivalent to 0.36 litres per minute. This is nearly five times slower than the 1.67 litres per minute the JT-DC3L datasheet promises at zero head, the difference being delivery head and tubing resistance. A sampling period of 200 milliseconds therefore satisfies the condition by roughly three orders of magnitude. The group records both figures deliberately: an earlier draft quoted 0.6 centimetres per second from an unchecked estimate, the datasheet implies 0.28, and the bench measures 0.06. Only the last of the three was used to size the timing thresholds of Section 6.3, because a threshold derived from a datasheet the hardware does not obey will fire during normal operation. The ESP32 digitises voltage using a successive approximation register converter [1, 4], and mapping a continuous range onto finite integer steps introduces an unavoidable loss of precision with a quantisation step of

$$\Delta V = \frac{V_{ref}}{2^N} \quad (2.1)$$

and quantisation noise bounded within $\pm \frac{1}{2}\Delta V$. This error is mathematical in nature and can never be filtered away in software [1]. At 12 bits and 3.3 volts the step is 0.806 millivolts, a figure used in Section 3.3. The lectures also warn that the transfer curve is severely nonlinear near both ends of the

range, which is why our dividers place each signal near the middle of the span.

A microcontroller operating at 3.3 volts with roughly 12 milliamperes per pin cannot perform mechanical work directly, so a power stage with galvanic separation has to be inserted. When current through an inductive load is interrupted abruptly, the collapsing field produces a reverse spike following Lenz law, $V = L di/dt$, which can exceed several hundred volts and destroy the switching device; the standard remedy is a flyback diode placed antiparallel across the load [1].

2.4 Application layer messaging

A standard HTTP header easily exceeds 500 to 1000 bytes, so sending a measurement of a few bytes spends most of the energy budget on framing, and every transaction additionally forces the radio through a complete TCP handshake [5]. MQTT addresses this with a binary frame whose fixed header is two bytes, routed through a broker under a publish and subscribe model that decouples communication in space, time and synchronisation [5, 6].

Three MQTT mechanisms are exploited directly by our system. The three quality of service levels define delivery as at most once, at least once and exactly once, and the lectures caution against defaulting every topic to the highest level because a constrained node will exhaust its memory. The last will mechanism lets the broker publish a pre registered payload by itself once a client vanishes without a clean disconnect, and retained messages make the broker push the current value immediately to a newly subscribing client, eliminating the cold start gap [5]. On the security side, MQTTS carries the same binary frames inside a TLS tunnel on port 8883 instead of the unencrypted port 1883, and the lectures stress that a username and password pair alone is insufficient without an access control list mapping each client to the topics it may read and write [5, 7].

2.5 Data infrastructure and the feedback sync principle

Machine telemetry has three structural invariants: it is append only, indexed primarily by time, and its write to read ratio is very high [8]. A relational engine is built for transactional integrity, so every inserted row forces a rebalancing of an in memory B-tree, whereas a time series engine optimises for tail appends and separates indexed tags from unindexed fields. The lectures also warn about index cardinality explosion when a randomly varying value is placed in a tag position [8], a principle our schema in Section 5.2 follows. The REST style demands client server separation and statelessness, favours nouns over verbs, and asks the designer to evaluate safety $f(s) = s$ and idempotency $f(f(s)) = f(s)$ [9, 10]; on the microcontroller side, firmware must forbid dynamic allocation inside the loop and reserve a fixed buffer at startup [10, 11].

The single idea with the largest influence on our interface design is the feedback sync principle derived from the device shadow model [8]. The anti pattern to avoid is a dashboard toggle that flips to the on state the instant the user presses it, before anything is known about whether the real machine started.

The correct architecture splits the path: the user action maps to an outgoing command topic while the indicator stays unchanged until an explicit acknowledgement arrives from the device. This is the basis for the mechanism described in Section 4.3.

2.6 On off control with hysteresis

The material in this section does not appear in the course slides. The group added it from control engineering literature and states the sources here explicitly, so that it is clearly distinguished from the knowledge taken from the lectures.

On off control, also called two position control, is the simplest form of feedback control, in which the actuator takes one of exactly two states [12]. If the two thresholds coincide, even the smallest measurement noise around that point makes the actuator flip repeatedly, causing rapid wear and repeated inrush events. The standard remedy is to separate them into a hysteresis band inside which the state is held unchanged [13]:

$$u(t) = 1 \text{ if } h < h_L; \quad u(t) = 0 \text{ if } h > h_H; \quad u(t) = u(t^-) \text{ if } h_L \leq h \leq h_H \quad (2.2)$$

The width $h_H - h_L$ is a direct trade off parameter: a wider band lowers the switching frequency but widens the level excursion.

For diagnosis, the group applies the principle of checking consistency between two independent signal sources, following established fault diagnosis theory [14]. Slow leaks belong to the class of small persistent shift detection, for which the cumulative sum method of Page is far more sensitive than a fixed threshold because it accumulates small deviations over time [15].

To select control parameters before hardware existed, the group built a dynamic model from the volume balance $dV/dt = Q_{in} - Q_{out} - Q_{leak}$ with $h = V/A$, where discharge follows Torricelli law $Q_{out} = C_d A_o \sqrt{2gh}$ [16]. The square root dependence makes the plant nonlinear, since discharge at a full tank is appreciably faster than near empty. This is why the hysteresis parameters had to be swept in simulation, as described in Section 8.3.

Chapter 3

ARCHITECTURE AND HARDWARE DESIGN

3.1 Overall architecture

The system consists of four blocks in series: an edge node built around an ESP32 with its sensors and one actuator; an MQTT broker acting as a content based message router; a backend service comprising an ingestion process, a database and an API server; and a web dashboard. A fifth, optional element is an outbound tunnel that publishes the dashboard on a public HTTPS address so that the tank can be supervised from anywhere with Internet access, not only from inside the laboratory network.

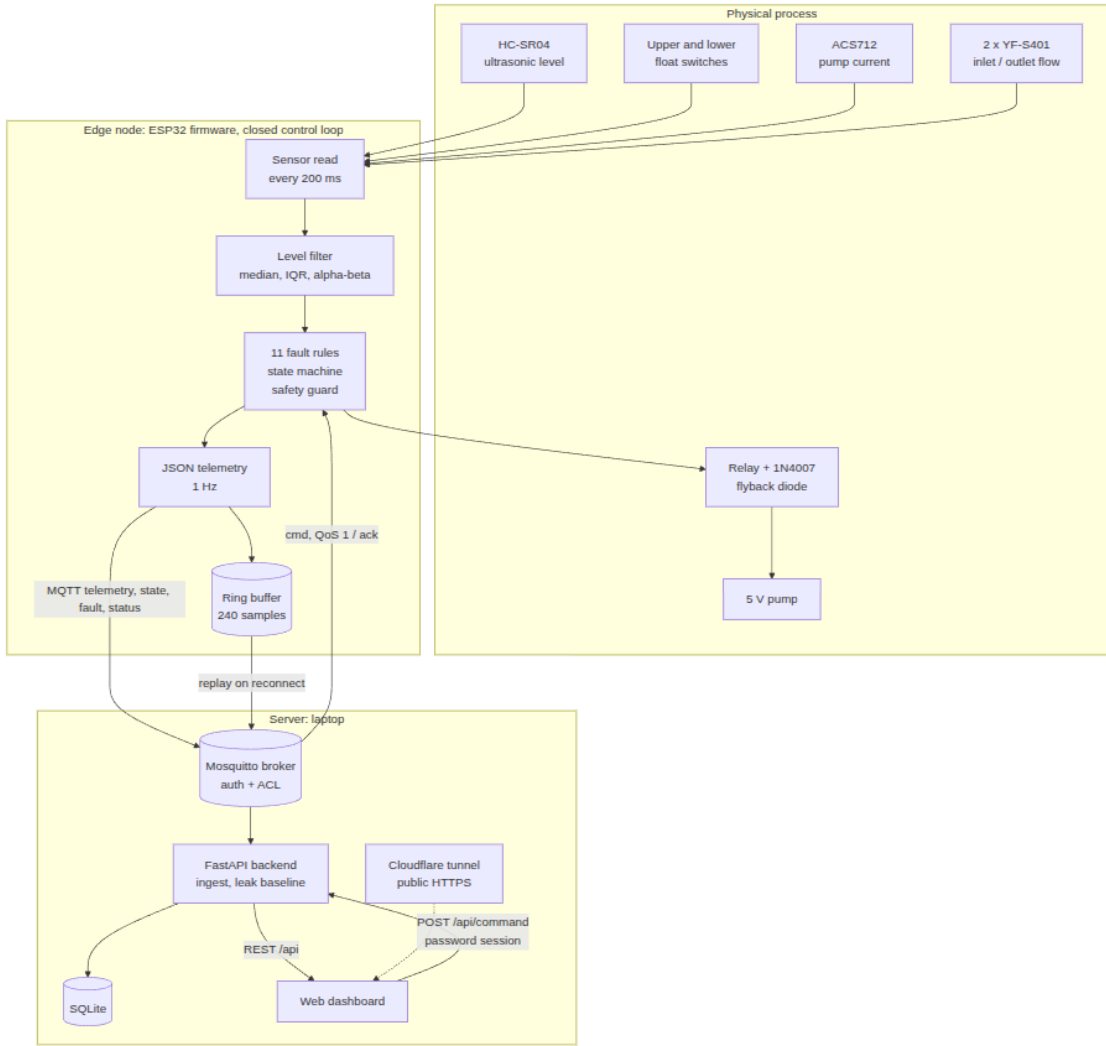


Figure 3.1.1: Architecture of the smart water tank system. The closed control loop lives entirely inside the edge node.

Mapping onto the five layer model of Section 2.2: the perception layer is the ESP32 with its sensors and relay; the transport layer is a Mosquitto broker over Wi-Fi; the processing layer is the ingestion process, SQLite and a FastAPI server; the application layer is the dashboard; and the business layer, kept deliberately minimal at this scale, is the daily volume report. The single most important property of Figure 3.1.1 is that the closed control loop lives entirely inside the first block, so the remaining blocks never sit on the path of a pump switching decision.

Wi-Fi was selected over BLE, Zigbee and LoRaWAN for three reasons. The system is mains powered, so the low energy advantage of BLE and LoRaWAN carries no weight. The device sits indoors within a few tens of metres, comfortably inside Wi-Fi coverage. Most importantly, Wi-Fi gives the node a native IP address so it speaks to the broker directly, whereas every alternative needs an edge gateway to translate between two protocol worlds [17], adding hardware and a point of failure. The conclusion reverses for tanks spread over a wide area or for a battery powered node. One practical constraint surfaced on the bench: the ESP32 radio supports only the 2.4 GHz band, so the node cannot join a 5 GHz only access point, and the firmware therefore accepts up to three network credentials and tries them in turn.

3.2 Function placement and edge autonomy

This is the central architectural decision of the project. The assignment requires that level control keep working during a network outage. Combined with the placement rule of Section 2.2, which states that any operation needing an immediate safe stop must be computed at the edge [1], the group adopted a single principle: **the server never issues a direct pump switching command in automatic mode, and never bypasses the device’s own checks in manual mode.**

The edge node therefore owns sensor reading and filtering, the control state machine, every safety interlock, all fault detection rules, the pump health classifier and volume accumulation, because each of these demands a deterministic cycle and has to survive an outage. The backend owns only the things the node cannot do well: durable history, the daily volume report, and statistical baseline leak detection, which needs a window of several days and exceeds the memory available on the microcontroller.

This split makes the system degrade in a controlled way when connectivity is lost: the user loses remote visibility and remote commands, but the tank is still refilled at the correct thresholds and every safety constraint is still enforced. The behaviour is verified experimentally in Section 8.5.

3.3 Sensor selection and signal conditioning

Table 3.3.1: Main bill of materials.

Component	Role
ESP32 DevKit V1	Edge microcontroller
HC-SR04 ultrasonic sensor	Tank level, mounted 17.5 cm above the tank floor
Two YF-S401 Hall turbine flow sensors	Inlet (pump to tank) and outlet (tank to consumer) flow
ACS712 Hall current sensor, 5 A variant	Pump current, used for the pump health classifier
Two mechanical float switches on the storage tank	Upper level overflow guard and lower level fill permission
Single channel relay module, 5 V submersible pump	Switching stage and actuator
1N4007 diode, 100 nF ceramic, 1000 μ F electrolytic	Inductive protection, brush noise filtering, inrush buffering
10 k Ω , 20 k Ω dividers, 4.7 k Ω pull ups	Level shifting of 5 V signals, pull ups on the open collector flow outputs
12 V supply, 2 A fuse, 5 V 5 A buck converter	System power, distributed in a star from the converter output

The tank. The storage tank has a square base of 10 by 10 centimetres, so one centimetre of level is exactly 0.1 litres. The working range is 14 centimetres, which the firmware treats as 100 percent, giving a usable volume of 1.4 litres. The small volume is deliberate: it makes a full fill and drain cycle last about four minutes, so dozens of control cycles can be observed in one bench session.

Level sensing. Non contact ultrasonic measurement was chosen because the transducer never touches the water, so it neither corrodes nor accumulates deposits, and its output does not depend on water conductivity. Two properties of this particular installation shaped the software in Chapter 6. First, the beam of an HC-SR04 spreads over roughly 15 degrees and is already about 8 centimetres wide when it reaches the water, against a 10 centimetre tank, so the edge of the beam strikes the wall on every ping and a fraction of echoes return from the wall or the floor rather than the surface. Second, the sensor occasionally locks up after a supply disturbance and stays silent until its supply is cycled, which no firmware reset can reach. The first property is handled by a robust filter; the second by a timeout rule and a documented recovery procedure.

Echo line. The echo line was first routed through a TXS0108E automatic direction level translator. That part is designed for open drain buses and its internal pull ups and edge accelerators distorted the echo pulse, whose width is the only carrier of information. Replacing it with a plain 10 k Ω and 20 k Ω divider removed a 28 to 30 percent echo loss, as recorded in Section 8.8.

Flow sensing. Both inlet and outlet carry a YF-S401 turbine sensor, where pulse frequency relates to flow as $Q = f/K$. The bench revealed two problems that no datasheet mentions. The measured pump delivery is 0.36 litres per minute and the outlet draw is 0.09 to 0.15 litres per minute, while the YF-S401 is specified from 0.3 litres per minute: the inlet sits at the very bottom of the span and the outlet below it, where the turbine simply does not turn. Worse, while the pump runs both signal lines carry about 1500 hertz of interference conducted along the shared supply and ground, which a flyback diode and physical separation of the wiring did not reduce. The firmware therefore offers two sources of flow, selected at compile time: the pulse counters, or flow derived from the level rate described in Section 6.2. The prototype runs with the second. This is stated here plainly because the assignment lists the flow sensor as mandatory: the sensors are fitted, wired, pulled up with 4.7 k Ω resistors and read by interrupt, but the numbers they produce in this installation cannot be trusted and are not used for control.

Float switches. The two floats are not there to measure level. The upper float forms an overflow guard that operates independently of the ultrasonic path, and the lower float gives the pump a second, independent permission to start when the tank is low. Because two independent sources exist, the firmware can detect a contradiction between them and conclude that one reading is wrong. This is how the system satisfies the requirement to handle implausible sensor readings, as detailed in Section 6.4. Float polarity was measured rather than assumed, because a float wired with the opposite sense reads as permanently engaged.

Current sensing. The ACS712 output passes through a one to one divider that moves its no load point from 2.5 volts to 1.25 volts, the middle of the converter span. The 5 ampere variant produces 185 millivolts per ampere, which becomes 92.5 millivolts per ampere at the pin, or 10.81 milliamperes per millivolt. The firmware subtracts a startup offset averaged over 200 samples, reports the result in

milliamperes, and reports zero below a 4 millivolt noise floor, since with the pump off the raw reading wanders between 0 and 2.7 millivolts, a single converter step being 0.8 millivolts. The measurement cannot resolve currents below about 40 milliamperes, which is enough to separate a running pump from a stopped one but not to characterise the motor, and Section 8.7 returns to this.

3.4 Power stage, supply and mechanical build

The pump is switched through a relay module. A power MOSFET was considered, since the lectures favour it in edge designs of modest power thanks to its very low on resistance [1], but the relay was chosen because a pump running in two states at a low switching rate never exercises the frequency advantage of a MOSFET, and because the module drives the load from its own supply. Inductive protection uses a 1N4007 diode antiparallel across the pump terminals, alongside a 100 nF ceramic capacitor at the motor terminals whose purpose is not spike suppression but damping of brush noise. The relay polarity was determined by measurement: the module fitted is active high, and an earlier configuration that assumed active low made every stop command start the pump, as recorded in Section 8.8. The relay control pin is driven to the off level as the very first statement of the firmware, before the serial port or any sensor is initialised.

The system runs from a 12 volt supply through a 2 ampere fuse and a 5 volt, 5 ampere buck converter shared by the pump and the logic. Because the rail is shared, distribution matters: every load branch leaves the converter output terminal in a star, and the pump ground does not share a conductor with the controller ground. A 1000 microfarad capacitor near the controller supply pin buffers the inrush current of the motor, whose start peak would otherwise pull the rail below the brownout threshold and reset the chip [1].

The rig forms a closed water loop: a source bucket, the pump, the inlet flow sensor, the transparent main tank, a manual valve acting as the consumption load, and the outlet flow sensor. The ultrasonic bracket must be rigid, because vibration makes readings oscillate and would be misdiagnosed as a sensor fault. The circuit was assembled in steps, each with its own test program in the repository (`test_level`, `test_relay`, `test_flow`, `test_current`, `test_floats` and others) that had to pass before moving on.

Chapter 4

PROTOCOL AND COMMUNICATION

4.1 Design principles and topic tree

Three principles govern the protocol specification. First, every message carries a device generated timestamp, because although TCP preserves ordering on a single socket, brokers process multiple threads concurrently so final arrival order may differ, and deterministic ordering must therefore be imposed at application level inside the payload itself [5]. Second, the quality of service level is chosen per topic according to the consequence of losing a message. Third, every downward command must be answered by an upward acknowledgement carrying the same identifier.

The topic root is built from broad scope down to the individual device as `wt/lab1/esp32-01`, following the hierarchical naming rule given in the lectures [5].

Table 4.1.1: MQTT topic tree and the reasoning behind each setting.

Topic	QoS	Retain	Reasoning
<code>.../telemetry</code>	0	No	One lost sample in a continuous 1 Hz stream changes no conclusion about the trend, and in exchange latency is lowest with no device buffer
<code>.../state/pump</code>	1	Yes	Published only on change, so a loss leaves the dashboard wrong until the next transition
<code>.../event/fault</code>	1	No	Losing an alarm is incorrect behaviour for a safety function
<code>.../status</code>	1	Yes	The last will payload is published once, with no retransmission
<code>.../cmd</code> and <code>.../cmd/ack</code>	1	No	A lost command or acknowledgement is a direct functional failure

The design respects all three naming pitfalls listed in the lectures [5]: no leading separator, no whitespace inside the path, and six explicit subscriptions rather than a root wildcard. No topic uses the highest quality of service level, because it requires a four step handshake and packet identifier tracking on the edge device, while the duplicate delivery problem it solves is already handled at application level: each command carries a unique identifier, so a duplicate simply causes the device to resend the same acknowledgement without acting twice. This is the idempotency property of Section 2.5.

4.2 Message structure and fault codes

Payloads use JSON built with a statically allocated document, following the rule that forbids dynamic allocation inside the loop [10, 11].

```
{ "dev":"esp32-01", "ts":1790160600, "ts_ms":1790160600123, "seq":10421,
  "level_pct":62.4, "level_cm":8.74, "level_ok":true,
  "level_rate_cms":0.036, "level_model_cm":8.71, "blind_ms":0,
  "flow_lpm":0.36, "volume_l":143.62, "volume_today_l":12.40,
  "flow_out_lpm":0.16, "volume_out_l":98.10, "flow_src":"level",
  "pump":true, "state":"FILLING", "mode":"AUTO",
  "current_mv":36.1, "current_ma":390.2, "pump_health":"ok",
  "float_max":false, "float_min":false, "fault":"","rssi":-55,
  "lv_try":605, "lv_ok":563, "rj_spread":31, "ctl_gap_ms":201 }
```

Listing 4.1: Telemetry message payload, abridged.

The `seq` field is a monotonically increasing counter allowing lost messages to be counted exactly in Section 8.5. The `ts_ms` field is produced by the device after time synchronisation and supports latency measurement. The `level_ok` field states whether the reading can be trusted, letting the interface distinguish a genuinely low tank from a broken sensor. The `flow_src` field states whether the flow figures come from the pulse sensors or are derived from the level, so no consumer can mistake one for the other. The `lv_try`, `lv_ok` and `rj_*` counters record how many level pings were attempted, accepted and rejected at each filter stage; they turned out to be the most useful diagnostic in the project, as Section 8.8 shows. The `ctl_gap_ms` field reports the longest control period since the previous live message, so the first message after an outage proves whether the control loop was ever stalled by the network. Messages replayed after reconnection carry a `replay` flag and a reduced field set, and are excluded from latency statistics. The full message runs to several hundred bytes, so the serialisation buffer is 1024 bytes and the MQTT client buffer 1200 bytes; Section 8.8 explains why both margins are explicit.

Eleven fault codes are generated on the device and one on the backend. They fall into three groups. The self clearing group, `SENSOR_TIMEOUT`, `SENSOR_CONFLICT` and `LEVEL_LOST`, concerns signal quality and resolves on its own once the signal has been valid for long enough. The latching group, `OVERFLOW`, `NO_CURRENT`, `DRY_RUN`, `NO_PROGRESS`, `FILL_TIMEOUT`, `HARD_LIMIT` and `VOLUME_LIMIT`, requires the operator to confirm that the physical cause has been dealt with before the fault can be cleared. The warning group, `LEAK_SUSPECTED` on the device and `LEAK_BASELINE` on the backend, raises an alert without locking the pump. Every fault message also carries the pump health label and the pump current at the instant

of the fault, captured before the pump is stopped, so the record states not only that something failed but which side of the pump failed. Trigger conditions are given in Section 6.4.

4.3 Command and acknowledgement mechanism

This section implements the feedback sync principle of Section 2.5 and satisfies the requirement that commands be acknowledged with a verified state.

```
// .../cmd
{"cmd_id":"a7f3c2e1","action":"pump","value":true,"ts":1735689600}
// .../cmd/ack
{"dev":"esp32-01","cmd_id":"a7f3c2e1","status":"rejected",
 "reason":"overflow_guard","state":"OVERFLOW_LOCK","pump":false}
```

Listing 4.2: Command message and matching acknowledgement.

The acknowledgement reports not only the outcome but also the actual device state after processing, so the dashboard always displays a state the device has confirmed rather than one the user has just requested. Four actions are supported: change mode, switch the pump, clear a fault and reset the volume counters. Ten rejection reasons are defined: `auto_mode_active`, `fault_active`, `overflow_guard`, `float_max_triggered`, `water_too_close`, `level_sensor_invalid`, `min_off_time`, `no_active_fault`, `condition_still_present` and `unknown_action`. The dashboard translates each into a plain sentence, and for `min_off_time` it shows a countdown on the pump button until a start will be accepted.

The rejection table is worth more than a debugging aid: during the demonstration, pressing the pump on button while a fault is active and seeing an explicit rejection with a named reason is visible proof that manual mode cannot override the interlocks.

4.4 Connection lifecycle and availability monitoring

The firmware implements the four state connection machine described in the lectures. The critical point is the accompanying design note: connection logic must never sit inside a blocking wait loop, because that stalls the processor and suspends both analog sampling and safety checks for the whole outage [5]. Our reconnection routine therefore never blocks, retries only once its interval has elapsed, and the control loop runs independently of network state. The retry interval grows exponentially with an upper bound, $T_{retry} = \min(T_{max}, T_{base} \cdot 2^n)$ with T_{base} of 1 second and T_{max} of 30 seconds [18].

A non blocking design still has to account for the calls it cannot avoid. Opening a TCP connection to an unreachable broker blocks inside the network stack, and the default timeout of the ESP32 client is three seconds, during which the control loop does not run. The firmware therefore opens the socket itself with a 500 millisecond limit before handing it to the MQTT client, and bounds the wait for the broker's connection acknowledgement to two seconds. After a Wi-Fi reconnection the device also resolves the broker's host name over multicast DNS, because a laptop that moves between networks receives a different address on each; a fixed fallback address is used if the name does not resolve.

On connecting, the device pre registers a last will message announcing that it is offline, then overwrites it with an online message once the session is established. Should the node suffer an abrupt loss of power, the broker keep alive supervisor publishes the stored will by itself [5], so the interface switches to a disconnected state with no polling logic on the application side. A second backend check treats the device as offline when the newest telemetry message is older than 5 seconds. The two complement each other, since the will reacts quickly to a total power loss while the freshness check also catches a live session whose data stream has stopped.

Chapter 5

BACKEND, DATA MANAGEMENT AND INTERFACE

5.1 Backend architecture

The backend is a single Python process containing three concurrent components. The ingestion component holds a persistent broker session and subscribes to the six specified topics. The API server exposes endpoints for the dashboard and for extracting experimental data. The rolling baseline leak detector is invoked on every incoming telemetry message and is described in Section 6.6; because it learns slowly, it is rebuilt from the stored history each time the process starts, so a restart does not discard days of learning. In addition the process keeps an in memory copy of the latest state, so that the current state endpoint can answer without touching the database, since the dashboard calls it once per second.

For every telemetry message the ingestion component records the device timestamp alongside `recv_ts`, written by the backend on arrival, so that a one way delay $\tau = t_{recv} - t_{sample}$ is available from a single query without external instrumentation. The device timestamp is stored twice: `ts` at one second resolution, kept so that existing consumers of the payload continue to parse, and `ts_ms` at millisecond resolution, which is the field the latency query prefers. The coarser field alone carries a quantisation error of up to 500 milliseconds, larger than the transport delay it is meant to measure, and Section 8.5 records the corrected figures side by side. The measurement assumes both clocks are synchronised; messages emitted before synchronisation completes carry a zero timestamp and are excluded from the statistics. Because that assumption cannot be discharged on a microcontroller disciplined only by the network time protocol, the one way figure is treated as a comparison quantity and the command round trip, timed entirely by the server clock, is the reported result.

5.2 Data schema and API

The schema has six tables, split according to data behaviour, and is shown with the whole repository in the code guide delivered alongside this report: `telemetry` is written continuously at one row per second, while `faults`, `pump_events`, `commands`, `availability` and `config` are written sparsely and discretely. The telemetry table is indexed on the timestamp column, the only column ever used as a filter predicate. The group applies the tag and field separation principle of Section 2.5: the device identifier acts as an indexed tag while the measured quantities are unindexed fields. In particular the sequence number column, although it changes on every row, is deliberately left unindexed to avoid exactly the cardinality explosion the lectures warn about [8]. The command table stores both the send time and the acknowledgement time, whose difference is the command round trip latency.

On database choice, the group is aware that the theory in Section 2.5 identifies real limits of relational engines for high rate time series data [8]. Those limits, however, appear at thousands of writes per second, whereas this system writes one row per second from one node, three to four orders of magnitude below the problematic regime. Deploying a dedicated time series engine would add operational complexity without any measurable benefit, so the group chose SQLite while still respecting the schema design principles above [19].

Table 5.2.1: Main API endpoints.

Method	Path	Purpose
GET	<code>/api/latest</code>	Latest state, data freshness, pending commands
GET	<code>/api/telemetry</code>	Time series over a requested interval
GET	<code>/api/volume/daily</code>	Volume pumped in and drawn out per day
GET	<code>/api/leak/baseline</code>	State of the rolling baseline, slots learned
POST	<code>/api/login</code> , <code>/api/logout</code>	Open or close an administrator session
GET	<code>/api/auth</code>	Whether this browser may control the pump
GET	<code>/api/faults</code> , <code>/api/events</code> , <code>/api/commands</code>	Fault log, pump switching log, command log with acknowledgement latency
POST	<code>/api/command</code>	Issue a command, returns its identifier; session required
GET, PUT	<code>/api/config</code>	Read thresholds; update requires a session
GET	<code>/api/stats/latency</code>	Latency statistics used in Section 8.5

Method selection follows the two properties introduced in Section 2.5. Read endpoints use GET because they are safe and change no state. The configuration endpoint uses PUT because replacing the full threshold set is idempotent. The command endpoint uses POST because each call creates a new

command record with its own identifier and is therefore not idempotent [10].

Daily volume. The first implementation computed each day’s volume as the maximum minus the minimum of the accumulated counter. That is wrong whenever the counter is reset during the day, and it was: the result swung by litres after every reset. The corrected query uses a window function to take the increment between consecutive rows and sums only increments that are positive and physically possible in one second, so a reset contributes nothing instead of a large negative step. Inflow increments are additionally counted only while the previous row showed the pump running, because with flow derived from level the inlet cannot be non zero with the pump off. Rows stamped before the device clock synchronised carry a timestamp near zero and are assigned to the server’s arrival date instead, which removed a spurious day in 1970 from the chart. Figure 5.2.1 shows the result for the three bench days.

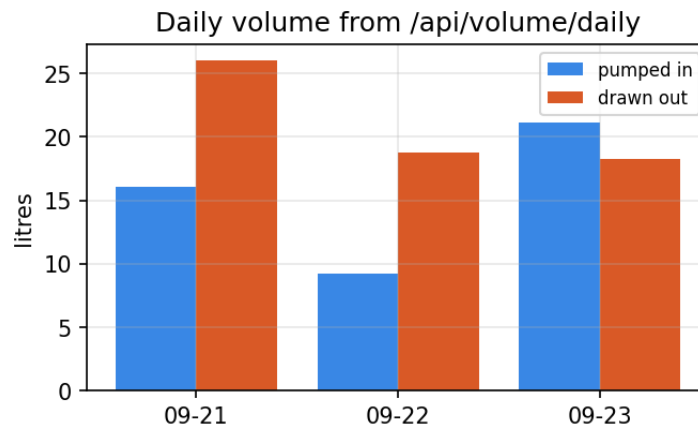


Figure 5.2.1: Daily volume pumped in and drawn out, as served by the API.

5.3 Monitoring interface

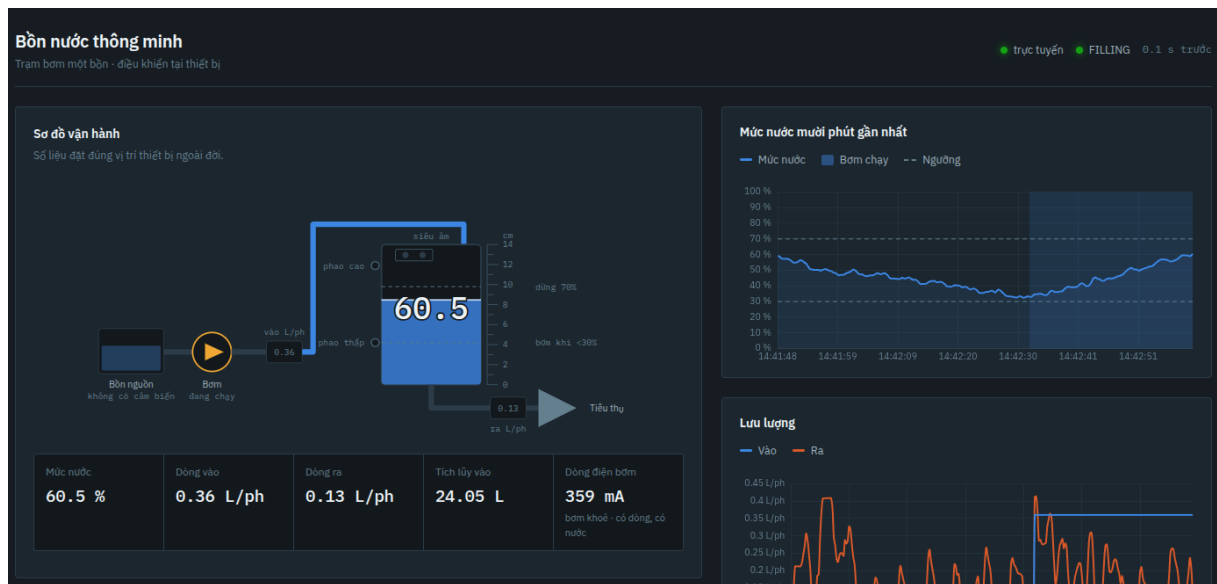


Figure 5.3.1: Monitoring dashboard during an automatic fill: process mimic with the tank at 60.5 percent, numeric tiles with the pump health label, and the level history. For this screenshot, recorded bench data from 23 September were replayed into an isolated copy of the backend, so the live database was not modified.

The interface is designed for an operator who glances at the screen for a few seconds to answer three questions: what is the level, is the pump running, and is anything wrong. It is a single HTML file with no build step and no framework. The upper part is a process mimic in the style of a control room: the source bucket, the pump, the pipe and the tank are drawn as a diagram, the water in the tank rises and falls with the measured level against a ruler graduated from 0 to 14 centimetres, and the low, high and overflow thresholds are drawn at their true positions for the active configuration. Drawing the thresholds on the tank lets a viewer see at once where the level sits relative to the hysteresis band and why the pump just switched.

Below the mimic are the numeric tiles: level, inlet and outlet flow, pump current in milliamperes with the pump health label underneath it, and today's volume. Beside them are the history charts, a daily volume bar chart, and three logs: faults (with the pump health at the moment of each fault), pump events and commands. A freshness line next to the connection lamp shows how many seconds ago the last message arrived.

Two display details matter more than they appear to. The device reports once per second, so a number written straight into the page moves in visible steps; the interface eases the displayed value towards each new reading. And the filtered level still moves by a few millimetres from sample to sample, which on a large readout looks like instability; the display applies a deadband of 0.25 centimetres, below the resolution anyone can act on, so the number holds still unless the water really moves. Neither detail touches the data: the database and the API carry the unmodified values.

On the update mechanism, the lectures recommend REST for low frequency administrative tasks and

WebSocket for live streams [18, 20]. The group chose polling at 1 Hz, which runs against that general recommendation and therefore needs justification. The device publishes at 1 Hz, so the polling rate equals the data production rate and no request is wasted. With a handful of clients and one node the resulting traffic is a few kilobytes per second. A WebSocket introduces a stateful channel with its own lifecycle to manage, including heartbeats and reconnection [18], that is a new set of failure states in exchange for a benefit that is not measurable at this scale. The conclusion reverses beyond roughly ten nodes or a required update rate of 5 Hz.

The feedback sync principle is implemented in how command state is displayed. When the user presses the pump button the interface does not change the pump symbol; it posts the request, receives an identifier, shows “sending command” and polls the command log for that identifier. When the acknowledgement arrives it reports the outcome in a plain sentence, including the reason for a rejection, while the pump symbol follows only the pump state reported in telemetry. An early version gave no feedback at all on a rejection, and a user reported that manual mode “did not work” when the device had in fact correctly refused a start inside the 20 second minimum off time; the countdown shown on the button now makes that interlock visible instead of mysterious.

Remote access and control rights. The assignment frames the system as remote supervision, so the dashboard is published through an outbound Cloudflare tunnel that gives it a public HTTPS address without opening any inbound port on the laboratory network. Anyone with the address can watch. Control is a separate matter: the control panel stays hidden until the browser holds an administrator session, obtained by entering a password kept in a file outside the repository. The group first tried to grant control by client address, allowing only the host machine, and abandoned it for two reasons found in testing: through the tunnel every request arrives from the loopback address, so the rule granted control to the whole Internet; and on the host itself every open tab could control the pump, which is not what the owner meant by “only my machine”. Section 7.2 describes the session mechanism.

The interface adapts to the state reported by the device, following the conditional rendering principle [18]: data older than 5 seconds or an arriving last will message turns the connection lamp red; a fault or overflow lock switches the status to an alarm style; and a false `level_ok` flag marks the level as untrusted. Hiding or disabling buttons is only a convenience, since the backend rejects commands without a session and the device rejects unsafe ones regardless of who sent them.

5.4 Device simulator

The simulator implements the hydraulic model introduced in Section 2.6 together with a sensor noise model, and publishes messages that conform to the specification of Chapter 4. It serves two purposes: while components had not arrived it allowed the backend and interface branches to be developed fully, and it allows control parameters to be swept across dozens of configurations in a few minutes, which is not feasible on real hardware. The sweep results are reported in Section 8.3. The simulator can also inject three failure modes, namely a pump that runs without delivering water, a sensor stuck at a fixed value, and a leak at a prescribed flow rate.

Chapter 6

CONTROL AND SYSTEM INTELLIGENCE

6.1 Overview

This chapter presents the main technical contribution of the project. All of it runs in the microcontroller firmware, with the single exception of the rolling baseline leak detector of Section 6.6, which sits on the backend for the reason given in Section 3.2. The control loop runs on a fixed 200 millisecond cycle of six steps: read the sensors, filter the level, update the parallel level model, evaluate the fault rules, run the state machine, and drive the outputs. The loop does not depend on network state; the only network work inside it is bounded as described in Section 4.4.

6.2 Computed quantities

Level. The ultrasonic sensor measures the distance d from the transducer to the water, and the level is $h = 17.5 - d$ centimetres, then corrected as $h_{corr} = a \cdot h + b$ with coefficients from the calibration of Section 8.2. Because a share of echoes in this narrow tank return from the wall or the floor (Section 3.3), a single ping is never trusted. Each 200 millisecond cycle fires one ping and the reading passes through a chain of gates, each of which counts its own rejections so that the chain can be tuned from evidence rather than intuition:

1. **Window and geometry.** The last fifteen pings are kept; pings outside the physical geometry of the tank are discarded.
2. **Spread.** If the interquartile range of the window exceeds 3.5 centimetres the window is rejected, because the sensor is then alternating between two surfaces and the median of the window means nothing. Section 8.8 explains why this is the interquartile range and not the full range.
3. **Median.** The median of the window becomes the candidate reading. A median tolerates isolated outliers that would drag a mean.
4. **Rate.** The candidate is rejected if it differs from the last accepted value by more than physics

allows in the elapsed time plus a 2 centimetre noise allowance, and, while the pump runs, if it has fallen by more than 1 centimetre, since the tank cannot drain faster than a running pump fills it.

5. **Alpha-beta filter.** Accepted readings feed a two state filter that tracks both level $\hat{h}$ and its rate of change $\hat{v}$:

$$\hat{h}^- = \hat{h} + \hat{v} \Delta t, \quad r = h - \hat{h}^-, \quad \hat{h} \leftarrow \hat{h}^- + \alpha r, \quad \hat{v} \leftarrow \hat{v} + \frac{\beta}{\Delta t} r \quad (6.1)$$

with $\alpha = 0.03$ and $\beta = \alpha^2/(2 - \alpha)$. The earlier exponential moving average lagged a rising level by a constant offset; the alpha-beta filter tracks a steady ramp without lag, and its rate estimate $\hat{v}$ is used below for flow and for pump diagnosis.

If no reading is accepted for 5 seconds the level is marked untrusted (`level_ok` false). Independently of all of the above, a raw distance guard reads the unfiltered distance on every ping and declares the water too close after three consecutive readings nearer than 4.5 centimetres to the transducer. That guard shares nothing with the filter chain, so a filter defect cannot disable it.

Parallel level model. During a fill the sensor sometimes produces no accepted reading for tens of seconds, and the earliest firmware stopped every such fill at about half the target. The firmware now runs a model of the tank in parallel with the measurement, integrating $dh/dt = (Q_{in} - Q_{out})/A$ with the calibrated pump delivery and the latest drain estimate, and pulling the model towards each accepted reading with a gain of 0.05. While the sensor is blind during a fill, the state machine controls on the model instead. The model is never allowed to replace the sensor for long: after 45 seconds without a single accepted reading the pump stops and `LEVEL_LOST` is raised, and a returning reading that disagrees with the model by more than 2.5 centimetres is treated as suspect rather than adopted. The overflow float, the raw distance guard and the run time limits keep working throughout the blind interval.

Flow. Section 3.3 explains why the turbine readings are not used in this installation. With flow derived from level, inlet flow is the calibrated pump delivery of 0.36 litres per minute while the pump runs and zero otherwise. Outlet flow is computed from the rate estimate as $Q_{out} = -\hat{v} \cdot A \cdot 60/1000$ litres per minute, but only when the pump has been off for at least 20 seconds and the surface has settled; the estimate is smoothed and then *held* while the pump runs. An earlier version computed outlet flow as pump delivery minus the net level rate at every instant. Because the delivery term switches instantly while the rate estimate lags, every pump start made the outlet curve jump by the full pump delivery and decay back, which on the dashboard looked as if consumption rose whenever the pump started. Holding the drain estimate removed the artefact and matches the physics: the valve setting, not the pump, determines the draw.

Volume. Accumulated volume is the numerical integral of flow,

$$V(t) = \sum_i Q_i \cdot \frac{\Delta t}{60}, \quad (6.2)$$

computed on the microcontroller rather than the server, because if it ran on the server every message lost during an outage would become water permanently unaccounted for. The totals are written to non volatile storage every 60 seconds, so a sudden power loss costs at most one minute. Section 8.2 measures the resulting volume error at -5 percent on average.

Current. Pump current is reported in milliamperes using the scale factor of Section 3.3, with readings under the 4 millivolt noise floor reported as zero.

6.3 State machine and hysteresis

Table 6.3.1: Controller state set.

State	Entry condition	Exit condition
BOOT	Power up	A valid level reading is available
IDLE	Pump off, no fault	Automatic mode and (level below 30 percent or lower float) and the safety guard allows a start
FILLING	Automatic start	Level above 70 percent after at least 3 s, or switch to manual
MANUAL_ON	Manual start accepted	Manual stop, return to automatic, or the safety guard objects
FAULT_SENSOR	Sensor fault raised	Valid, consistent signal for 2.5 continuous seconds
FAULT_DRYRUN	Pump fault raised	Operator clears the fault
OVERFLOW_LOCK	Overflow or run limit fault	Operator clears the fault and the overflow condition is gone

The reason for a state machine rather than a plain conditional expression is that fault states need to be sticky. A conditional rule would allow the pump to restart the moment a transient fault condition disappears, for instance when the suction tube touches the water again; the state machine holds the system in the fault state until an explicit intervention occurs. One invariant is enforced at the top of every cycle: whenever a fault code is set, the state must be a fault state. An early version let the manual and filling states return to idle without clearing the fault code, which left a device that looked idle and healthy while silently refusing every start.

The transition between **IDLE** and **FILLING** implements equation (2.2) with thresholds of 30 and 70 percent, leaving a 15 percent margin up to the 85 percent safety limit to absorb overshoot. Two timing constraints complement the band: a minimum on time of 3 seconds and a minimum off time of 20 seconds. Hysteresis rejects measurement noise near the threshold, while the timing constraints reject genuine short period oscillations and protect the pump from repeated inrush.

The minimum on time deserves a note, because the first draft set it to 10 seconds and that value was unsafe. At the measured 0.36 litres per minute a forced 10 second run adds 0.06 litres, over 4 percent of this 1.4 litre tank, while the state machine has no authority to stop it. Three seconds is long enough to stop the relay chattering and short enough that the band still governs. A hysteresis width of 40 percent may look wide for a control system, but the tank is a buffer: the goal is to guarantee water availability

while limiting pump starts, not to hold a set point.

6.4 Fault detection rules

The system implements eleven rules where the assignment asks for at least one. They are evaluated every cycle in priority order and the first rule that fires ends the chain. Firing a rule stops the pump, lights the fault lamp, and publishes a fault message.

Table 6.4.1: Fault detection rules and trigger conditions.

Code	Trigger condition	Action
SENSOR_TIMEOUT	No accepted level reading for 45 s	Sensor fault, self clearing
SENSOR_CONFLICT	Upper float engaged while level reads below 70 percent, or lower float engaged while level reads above 60 percent	Sensor fault, self clearing
OVERFLOW	Upper float, or raw distance guard, or level at or above 85 percent	Lock
DRY_RUN	Pulse flow below 0.25 L/min for 6 s with pump on; active only with sensor flow	Lock
NO_CURRENT	Pump commanded on, no current for 2 s	Lock
LEAK_SUSPECTED	Pump off yet inlet flow above 0.20 L/min for 60 s	Warning only
FILL_TIMEOUT	One run longer than 240 s	Lock
HARD_LIMIT	One run longer than 300 s, unconditionally	Lock
VOLUME_LIMIT	More than 2.5 L pumped in one run	Lock
NO_PROGRESS	Pump on 90 s without the level rising 1.5 cm	Lock
LEVEL_LOST	Filling blind on the model for more than 45 s	Sensor fault, self clearing

SENSOR_CONFLICT is a direct application of the independent source consistency principle of Section 2.6 [14]. With only an ultrasonic sensor the system cannot tell a right reading from a wrong one as long as the value stays plausible; a mechanical float turns a wrong reading into a detectable contradiction.

The assignment's example rule, pump on with near zero flow, appears twice. **DRY_RUN** implements it literally on the turbine signal, and is disabled when flow is derived from level, because inlet flow is then a constant that can never fall below any threshold: leaving the rule enabled would have been a guard that could not fire. **NO_PROGRESS** implements the same physical idea on a measured quantity: a pump that draws current while the level does not rise is not moving water. Its mark only ever moves down with noise and moves up only on real progress, so a single high outlier cannot hide a stalled fill. Section 8.8

records a bench episode in which the level reading was false for the whole of a fill and this rule, which never consults the validity flag, still stopped the pump.

The 45 second sensor timeout is long by design. The ultrasonic sensor in this tank drops out for tens of seconds during normal fills, and the parallel model bridges those gaps; a short timeout would stop every fill early, which the first version did at 4 seconds. The price is that a disconnected sensor is reported after 45 seconds rather than 4, which is acceptable because every overflow layer that does not rely on the sensor remains active meanwhile.

6.5 Manual mode and safety interlocks

The assignment requires that manual mode must not override safety constraints. The group meets this with an architectural decision inside the firmware: **every code path capable of starting the pump is forced through one single guard function.** The function returns nothing when starting is permitted and a reason code when it is blocked, checking in order: an active fault, the upper float, the raw distance guard, level at or above the safety limit, an untrusted level, and the minimum off time.

An earlier revision listed an empty source tank as a further condition. It was removed, and the removal is worth recording: the source vessel carries no instrument, and the condition was in fact reading the lower float of the *storage* tank under a wrong label. Blocking the pump on a quantity the system cannot observe is not a safety measure but a permanent refusal to operate, and dry running is covered by rules that rest on quantities the system does observe: absent current and absent level progress.

Because both the automatic state machine and user commands pass through the same guard, verifying the interlock means reviewing one function rather than auditing the codebase. When a command is refused, the acknowledgement carries the reason. Moreover, while in `MANUAL_ON`, the guard is re-evaluated on every cycle, so lifting the upper float stops a manually started pump within one 200 millisecond cycle, and the fault cannot be cleared while the float is still up: the clear command is answered with `condition_still_present`.

6.6 Leak and anomalous consumption detection

This is one of the two advanced options in the assignment. It is built as two mechanisms at two places in the system.

The edge mechanism is the `LEAK_SUSPECTED` rule: once the pump is off, the supply line should carry no flow at all. It runs locally and therefore survives an outage, but it only sees the pipe section carrying the sensor and needs enough flow to turn the turbine, so with flow derived from level it is inactive.

The backend mechanism detects a sustained abnormal consumption level, the small slow shift discussed in Section 2.6 [15]. The day is divided into 48 slots of 30 minutes, and each slot keeps a baseline mean and variance of the volume used in it, updated by an exponentially weighted moving average:

$$\mu_k \leftarrow \mu_k + \alpha(V_k - \mu_k), \quad \sigma_k^2 \leftarrow (1 - \alpha) [\sigma_k^2 + \alpha(V_k - \mu_k)^2] \quad (6.3)$$

with $\alpha = 0.2$. Once a slot has been seen on at least three days, an alert `LEAK_BASELINE` is raised when $V_k > \mu_k + 3\sigma_k$ holds for two consecutive slots. Slicing the day reflects that domestic consumption varies strongly with the hour, and the two slot requirement is an accumulation step in the spirit of the cumulative sum method [15], suppressing alarms from a single unusual draw.

Two safeguards were added after the detector learned from bad data. A slot in which the counter went backwards, because it was reset, or in which more water was recorded than the pump can deliver in 30 minutes, is skipped entirely rather than learned, since early noisy flow readings had accumulated thousands of phantom litres. And the baseline is rebuilt from the stored history at every backend start. The honest status is that with three days of bench data, few slots have reached the three day minimum, so the detector is built and tested but has not yet raised a genuine alert.

6.7 Pump health classification

The second advanced option in the assignment is to use the consistency between pump current and flow to classify pump faults. The firmware assigns the pump one of five labels on every cycle:

- **idle**: the pump is off.
- **starting**: on for less than 2 seconds, or less than 25 seconds with no evidence yet.
- **ok**: current is present and the water is moving, meaning the level rate $\hat{v}$ exceeds 0.010 centimetres per second (or the turbine reads flow, when that source is selected).
- **no_current**: commanded on but no current, pointing at the electrical side: an open wire, a failed relay, a dead motor.
- **no_flow**: current present but no water moving after 25 seconds, pointing at the hydraulic side: a dry suction, a blocked or detached pipe.

The label is diagnostic, not protective: stopping the pump remains the job of `NO_CURRENT` and `NO_PROGRESS`. Its value is that the same symptom, a tank that does not fill, is split into two causes that send the operator to two different places, and that the label is captured at the instant of every fault, before the pump is stopped, and stored with the fault record. The 25 second evidence window is chosen from the measured fill rate: at 0.036 centimetres per second of net rise, 25 seconds produce 0.9 centimetres, well clear of the filter noise.

Chapter 7

SECURITY AND RELIABILITY

7.1 Threat model and mitigations

Under the ITU-T Y.2060 model of Section 2.2, security is a cross cutting capability acting on every layer rather than a layer of its own [2], so the analysis is organised per layer.

Table 7.1.1: Threat scenarios and mitigations.

Attack scenario	Consequence	Mitigation
Connect to the broker and publish a pump on command	Overflow, pump damage	Mandatory authentication, per topic access control
Sniff traffic to harvest credentials	Full control takeover	TLS transport on port 8883
Forge telemetry so the system mis-reads the level	Incorrect pumping, possible overflow	Local interlock based on the mechanical float
Replay commands repeatedly to make the pump chatter	Mechanical pump damage	Minimum on and off times enforced on the device
Call the API directly, bypassing the web interface	Unauthorised commands	Every control endpoint checks an administrator session
Guess the administrator password through the public address	Control takeover	Five failures in five minutes lock the source address

The two middle rows are worth highlighting because they are mitigated not by cryptography but by the control design itself. The mechanical float is a physical constraint that no network message can override, and the minimum switching times are enforced in firmware, so an attacker may send any number of commands without making the pump toggle faster than the limit. This illustrates the principle that safety in a cyber physical system cannot be separated from its control design.

7.2 Authentication, authorisation and encryption

The Mosquitto configuration disables anonymous connections entirely and mandates both a password file and an access control list file [21]. The group applies least privilege: the edge device may only publish on the northbound topics under its own branch and may only subscribe to its own command topic, while the backend may publish on the command topic and read the whole northbound branch. The practical meaning is that if the edge device is compromised, the attacker still cannot use that device to issue control commands, because the device account has no write permission on the command topic. This is exactly the node compromise scenario the lectures raise when introducing access control lists [5].

On the API side the system distinguishes a viewer, who needs nothing and may call every read endpoint, from an administrator, who may also call the command and configuration endpoints. Administrator rights are obtained by password. The password lives in a file outside the repository with owner only permissions, and is compared in constant time. A correct password creates a random 256 bit session token, returned as a cookie marked `HttpOnly` so page scripts cannot read it, `SameSite Strict` so another site cannot make the browser send it, and `Secure` whenever the request arrived over HTTPS, which through the tunnel is always. Sessions last seven days and are held in memory, so restarting the backend logs everyone out, which the group accepts at this scale. A missing or expired session returns 401. Five wrong passwords within five minutes lock further attempts from that address for the rest of the window, with the client address taken from the tunnel's forwarded header only when the connection itself comes from the local machine, since any remote caller could forge that header.

The group records one design it rejected. Control was first granted to requests from the host machine's own address. Behind the tunnel every request, from anywhere, arrives from the loopback address, so the rule silently gave control to the whole Internet; and even without the tunnel it gave control to every browser tab on the host rather than to a person. Identity has to come from something the person knows, not from where the packet appears to come from.

The broker runs on the unencrypted port 1883 and listens only on the laboratory network; the public tunnel carries the dashboard over HTTPS and never exposes the broker. For a real deployment the group switches to MQTTS on port 8883 [5, 7]. The cost should be stated plainly: the TLS handshake requires the microcontroller to perform public key operations, which noticeably increases startup time and memory consumption. One principle holds in every configuration, namely that no secret ever appears inside the telemetry payload, so even a successful eavesdropper gains no credentials.

7.3 Reliability during a network outage

This is a mandatory requirement and also the most important property of the system. Four mechanisms are implemented.

First, the control loop runs on a fixed cycle, and the two network calls it cannot avoid are bounded: opening a socket to the broker gives up after 500 milliseconds instead of the default three seconds, and waiting for the broker's acknowledgement gives up after two seconds. During an outage the system

therefore loses only remote visibility while retaining every control and safety function, and the longest control period is published in the first message after reconnection so the claim can be checked rather than assumed.

Second, when the link is down telemetry messages are not discarded but packed into a compact structure and pushed into a circular buffer holding 240 entries, about four minutes of data. The circular buffer was chosen for exactly the reason given in the lectures, namely constant time insertion with no array shifting [1]. Each entry stores only the essential fields as scaled integers instead of a full JSON string. Once the link returns the whole buffer is replayed in order, tagged so it can be excluded from latency statistics.

Third, accumulated volume is written to non volatile storage and reloaded at startup, so a sudden power loss does not erase the consumption history.

Fourth, inside the initialisation routine and before any sensor is read, the relay control pin is driven to the pump off level, so every restart begins from a safe state. The integrated brownout detector acts as an additional protective layer: when the supply falls below the critical threshold, flash writes become unreliable and the core may execute corrupted instructions, so the hardware deliberately holds the device in a safe reset state until the voltage recovers [1].

Table 7.3.1: Independent layers of overflow protection.

Mechanism	Location	Still effective when
High threshold of the hysteresis band	State machine	Sensor is healthy
Safety limit at 85 percent	Fault rule	Sensor is healthy
Raw distance closer than 4.5 cm, three pings in a row	Fault rule, unfiltered	The filter chain is producing nonsense
No level progress in 90 seconds	Fault rule	Level reading is frozen or false
2.5 litres in one run; 240 s and unconditional 300 s run limits	Fault rules	Every level sensor has failed
Upper mechanical float	Digital input	Ultrasonic sensor has failed
Relay driven off as the first instruction at boot	Firmware start up	Controller has just reset

The value of this arrangement lies in the independence of the layers. The first two both rely on the filtered ultrasonic level and fail together if it fails, but the raw distance guard bypasses the filter, the run limits consult only the clock, and the float requires no measurement at all. The time limits are honestly the weakest layer: a 240 second run at the measured delivery adds 1.44 litres, about a whole tank, so on their own they bound the damage rather than prevent it, and they sit behind the float rather than replacing it. No single point of failure disables the overflow protection as a whole.

Chapter 8

EXPERIMENTS, RESULTS AND CONCLUSION

8.1 Principles and reference instruments

Three principles apply to the whole experimental programme: every measurement must have a reference independent of the system under evaluation; every experiment must be repeated enough times to yield a dispersion figure and not merely a mean; and results must be reported through standard statistical indicators rather than qualitative remarks [22, 23]. The references are a millimetre scale standing in the tank, the geometry of the tank itself (a 10 by 10 centimetre base, so one centimetre of level is 0.1 litres), a measuring jug, and the server clock. The indicators are the per point error $e_i = x_{meas,i} - x_{ref,i}$, the mean absolute error and the root mean square error.

All figures in this chapter come from the database of the running system or from two scripts in the repository that read it: `tools/analyze_experiments.py` for the automatic cycles, switching and latency, and `tools/bench.py` for the interactive tests (level calibration, fault injection with a timestamped mark, and network outage). The analysed session is 23 September, 16:48 to 18:12, 4870 telemetry messages on the final filter configuration, with 99.5 percent of messages carrying a trusted level and no fault raised in the window. The test procedures are written out step by step in `docs/TESTING.md` so that any member can repeat them.

Table 8.1.1: Experiment plan and status.

ID	Subject	Reference	Status
E1	Level calibration	Scale in the tank, 6 levels	Pending final run
E2	Flow and volume error	Tank geometry, measured drain	Done, 6 cycles
E3	Control response	Configured thresholds	Done, 6 cycles
E4	Switching frequency	Pump event log	Done, 84 min
E5	Fault detection delay	Timestamped manual mark	Sensor timeout done; others pending
E6	Network loss and recovery	Sequence numbers	Done on earlier firmware; rerun pending
E7	Telemetry and control latency	Server clock	Done, $n = 75$
E8	Control policy comparison	Simulator	Simulated
E9	Wi-Fi power saving	Server clock	Done

8.2 Sensor calibration

E1, level. The procedure is to switch to manual mode, fill the tank to about 2, 4, 6, 8, 10 and 12 centimetres read on the scale, and at each level let `bench.py level` average 20 seconds of accepted readings. The tool prints the error per point, the root mean square error and a fitted slope and offset for the firmware constants. The filter’s own repeatability is already known: with the water still, twenty consecutive readings spanned 3.87 to 3.89 centimetres, a standard deviation of 0.010 centimetres. The calibration run itself had not been completed on the final installation when this report was written, and Table 8.2.1 is left for it rather than filled from an earlier mounting.

Table 8.2.1: E1 level calibration against the scale.

Scale (cm)	Device (cm)	Std. dev. (cm)	Error (cm)
RMSE, fitted a, b			

E2, pump delivery and flow sensors. The pump delivery was calibrated from the level: with the outlet closed the level rose 1.2 centimetres in 20 seconds, that is 0.12 litres in 20 seconds or 0.36 litres

per minute, against 1.67 litres per minute in the datasheet at zero head. The turbine sensors could not be calibrated in the usual way, by counting pulses against a collected volume, because in this installation they do not count water. Table 8.2.2 shows why.

Table 8.2.2: YF-S401 pulse rates on the bench.

Condition	Inlet	Outlet
Pump off, internal pull ups only	0.26 to 0.51 L/min apparent flow; exactly 50.00 Hz on one channel	
Pump off, 4.7 k Ω external pull ups	0 Hz	0 Hz
Pump running, before flyback diode	1474 Hz	1548 Hz
Pump running, diode fitted and wiring separated	1487 Hz	1567 Hz
Pulse rate expected for the real 0.36 L/min	about 35 Hz	

The pull ups removed mains pickup completely. The pump interference, forty times the real signal, was unaffected by the diode and by moving the wiring apart, which places it in the shared supply and ground rather than in radiation. The outlet draw of 0.11 to 0.20 litres per minute is in any case below the 0.3 litre per minute starting flow of the sensor. Flow for control and accounting is therefore derived from the level (Section 6.2), and the result is judged by the volume error below.

E2, volume estimation error. For each automatic fill the device’s pumped volume is compared with a reference built only from the level and the tank geometry:

$$V_{ref} = A \Delta h + \bar{Q}_{drain} T_{fill}, \quad (8.1)$$

where Δh is the level rise over the fill, T_{fill} its duration, and $\bar{Q}_{drain}$ the drain rate measured independently by fitting the level slope with the pump off for 90 seconds before the fill and 90 seconds after it (skipping 20 seconds after the stop while the pipe empties). Averaging before and after matters because the drain follows Torricelli’s law and is faster when the tank is fuller. The device value is the constant delivery integrated over the run, so the comparison tests the calibrated constant and the timing, not a sensor.

Table 8.2.3: E2 and E3: complete automatic fill cycles, 23 September.

Start	Stop (%)	Peak (%)	Over-shoot	Fill (s)	Drain (L/min)	V_{dev} (L)	V_{ref} (L)	Error
16:56:37	70.5	71.9	+1.47	237.4	0.180	1.289	1.520	−15.2%
17:09:56	70.8	72.4	+1.61	144.0	0.189	0.859	0.983	−12.6%
17:17:13	70.4	71.6	+1.20	146.0	0.146	0.872	0.875	−0.3%
17:32:34	70.3	71.5	+1.23	219.0	0.113	1.311	1.212	+8.2%
17:55:56	70.1	71.9	+1.85	144.0	0.196	0.864	1.006	−14.1%
18:02:00	70.4	71.5	+1.10	160.0	0.150	0.958	0.930	+3.0%
Mean	70.4		+1.41	175.1	0.16			−5.2%

The mean volume error is -5.2 percent with a spread from -15.2 to $+8.2$ percent. The sign pattern is informative: the three largest negative errors coincide with the three largest drain rates, which is what a constant delivery predicts, since a fuller source bucket and a shorter delivery head both change the real pump output while the model keeps 0.36. The error is therefore a property of treating delivery as a constant, and a working inlet flow sensor is the remedy.

8.3 Control response and switching

E3, control response. Figure 8.3.1 shows the analysed session. Every automatic fill stopped between 70.1 and 70.8 percent against a 70 percent threshold, and the level peaked 1.1 to 1.85 percent higher, the water still in the pipe after the relay opens. The worst peak, 72.4 percent, leaves 12.6 percent of margin to the 85 percent overflow limit. A fill from the lower threshold took 144 to 160 seconds at a net rise of 0.033 to 0.037 centimetres per second.

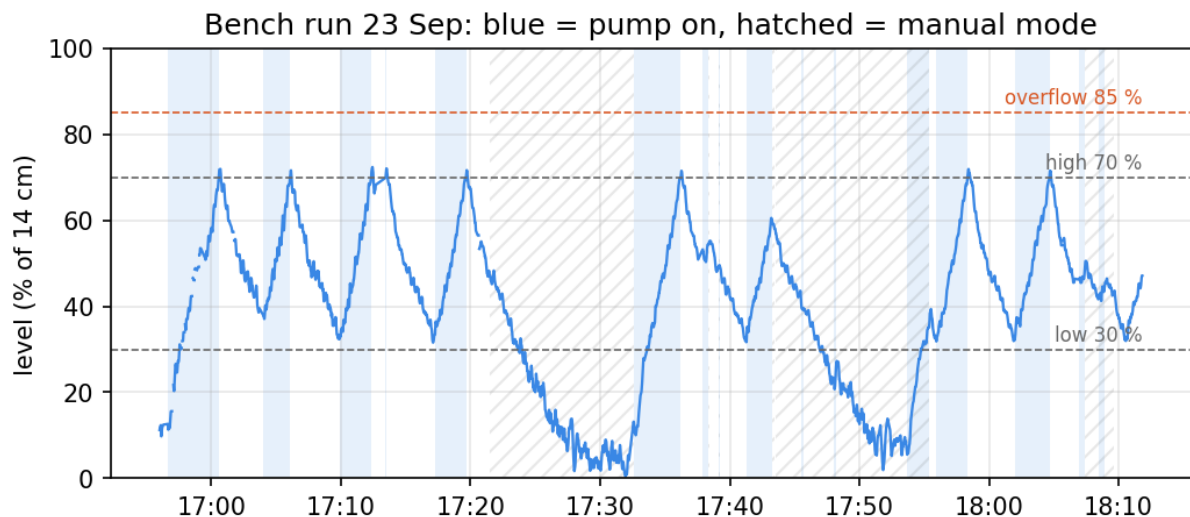


Figure 8.3.1: Level during the analysed session. Blue bands: pump running. Hatched: manual mode, used for command tests and for draining the tank below the band on purpose.

One result was not predicted. The automatic fills did not start at 30 percent but at 32 to 33 percent,

and in every case the lower float engaged in the same second. The float is mounted slightly higher than the 30 percent line, about 4.5 centimetres, and because the controller accepts either the float or the ultrasonic threshold as permission to start, the float wins. This is the designed behaviour of two independent start paths, and the finding is that the effective lower threshold is set by a screw on the tank wall rather than by the configuration.

E4, switching frequency. Over the 84 minute session the pump started 18 times, 12.9 per hour, including starts commanded by hand during the command tests. The shortest gap between a stop and the next start was 32 seconds, above the 20 second minimum off time, so the relay never chattered. The level was trusted in 99.5 percent of messages, and no fault fired, which bounds the false alarm rate of all eleven rules at zero in 84 minutes of normal operation.

A fault found by this analysis. Separating automatic from manual starts exposed pump starts that belonged to neither: starts at 38, 46 and 51 percent with the device in automatic mode and no command sent. Each coincided with a device restart, identified by the sequence number returning to one, and in the replayed history the pump turned on in the second message after boot with the level reading exactly zero. The cause was an initialisation order defect. At start up the firmware sets the time of the last valid reading to the current time, so that the sensor timeout counts from power up; but the validity flag was derived from that same timestamp, so for the first seconds after every boot the level was declared valid while still holding its initial value of zero. The state machine saw an empty tank and started the pump, which then ran until the real level passed 70 percent. The fix requires at least one accepted reading before the level can be declared valid, and the status message now carries the reason for the last restart, so a restart caused by a supply dip can be told apart from a firmware upload.

E8, control policy comparison. To show that the band and timing constraints matter, configuration A (49 and 51 percent, no timing constraints, approximating a single threshold) was compared with configuration B (30 and 70 percent with both timing constraints) on the simulator of Section 5.4, under the same ten minute continuous draw repeated three times, with one fixed target band from 25 to 85 percent for both. A switched 472 times per hour and B 8 times per hour, a reduction of 98.3 percent, with neither leaving the target band. The hardware session above, at 12.9 starts per hour with manual starts included and a different draw rate, is of the same order as the simulated figure for B.

8.4 Fault detection

E5, procedure. Each fault is injected by hand while `bench.py fault` is running; the operator presses Enter at the instant of the action, and the tool records the time to the first fault message, the fault code, the pump health label and the time until the pump state message reports the pump off. Table 8.4.1 lists the injections, the design delay and the measured result.

Table 8.4.1: E5 fault injection.

Injection	Expected code	Design	Measured
Unplug the echo wire	SENSOR_TIMEOUT	45 s	40.2 to 41.0 s after the last trusted message, $n = 9$
Lift the upper float while filling	OVERFLOW	0.2 s	
Lift the suction out of the water	no_flow, then NO_PROGRESS	25 s, 90 s	
Disconnect one pump wire	NO_CURRENT, no_current	2 s	
Manual start with upper float lifted	rejected, fault_active	immediate	

The sensor timeout figure comes from the database and is consistent with the design once the pipeline is accounted for: the level is marked untrusted 5 seconds after the last accepted reading, and the timeout counts 45 seconds from that same reading, so the fault arrives about 40 seconds after the last message that still showed a trusted level. Under earlier, shorter timeouts the same measurement gave 0.8 to 18 seconds, and the frequency of those faults during ordinary fills is how the group discovered that a short timeout was stopping normal operation.

8.5 Latency and behaviour during an outage

E7, and why the headline number is the round trip. The group reports the command round trip as the primary latency result, not the one way telemetry delay. A one way delay is $t_{recv} - t_{send}$ read from two different clocks, so it carries their offset as a systematic error, and the network time protocol on a microcontroller over Wi-Fi is accurate only to tens of milliseconds, the same order as the quantity measured. Immediately after a reboot the group observed an apparent one way delay of about 1100 milliseconds while the round trip at the same moment was 187 milliseconds, direct evidence that the one way figure was reporting clock skew. The round trip is stamped by the server at both ends, so the offset cancels exactly.

A measurement defect found and corrected. The first implementation carried a device timestamp at one second resolution, whose quantisation error of up to 500 milliseconds exceeded the delay being measured: the median of 844.6 milliseconds over 372 samples was an artefact of the field width. A millisecond field was added alongside, and the same measurement then gave 245.0 milliseconds. The resolution of an instrument must be finer than the effect it is meant to resolve, otherwise the instrument measures itself.

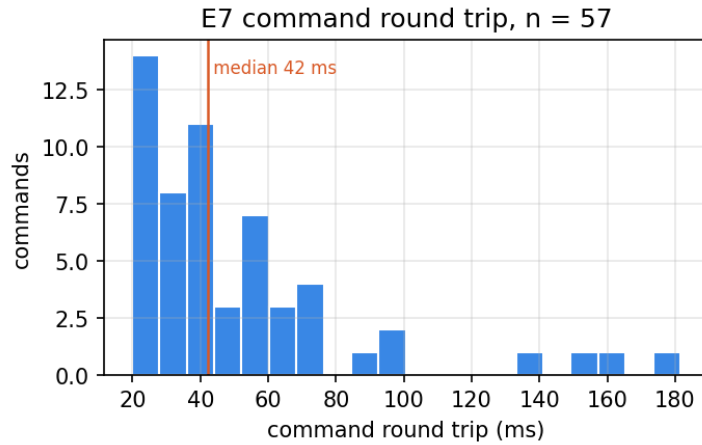


Figure 8.5.1: Distribution of command round trip times in the analysed session.

Table 8.5.1: Latency and outage results.

Quantity	Result
E7: command round trip, final firmware, median, 95th percentile, maximum (ms) (result of record)	42.4, 137.8 and 181.6, $n = 75$
E7: telemetry one way, final firmware, median and 95th percentile (ms)	166.9 and 238.0, $n = 4826$, comparison only
E7: command round trip, first firmware (ms)	225.8 and 575.3, $n = 14$
E7: telemetry one way at second resolution (ms)	844.6, $n = 372$, quantisation artefact
E6: outage duration, device isolated	60.0 s
E6: did the pump start on its own during the outage	no, it stayed off throughout
E6: messages buffered and replayed	32, sequence 598 to 629, 240 entry buffer
E6: messages lost	10, sequence 588 to 597

The round trip fell from a median of 226 to 42 milliseconds between the first and the final firmware. Section 8.6 accounts for most of it, the removal of Wi-Fi power saving. The final session also ran on a different access point, and the group has not separated the remaining improvement between the two causes.

E6, the outage procedure. The outage test isolates the device alone, leaving the broker and the backend connected to each other, because that is the failure the requirement describes. `bench.py outage` either drops the device’s traffic at the laptop firewall for a fixed interval or prompts the operator to switch the access point off and on, records every MQTT message, and reports the time until the broker published the last will, the time to reconnect, the messages replayed and lost, every pump transition contained in

the replayed history, and the longest control period during the outage. An earlier attempt stopped the broker process instead; that is not equivalent, because the device reconnected one second before the backend resubscribed and the whole replayed batch was published to a broker with no subscriber, so none of it reached storage.

Reading the outage result. Thirty two entries against a buffer of 240 means the buffer was about thirteen percent full, so a four minute outage remains within capacity. The ten lost messages deserve more attention than the thirty two that survived. They were published into a socket that the client still believed open: a connection severed by a silent packet drop is detected only when the 15 second keep alive expires, and every sample generated inside that window is written to a dead socket. The residual gap is bounded by the keep alive interval, not by the buffer size. The E6 figures above were recorded on the first firmware, in which a blocked broker could stall the control loop for three seconds per reconnection attempt; the final firmware bounds that stall to half a second and reports it, and the test is to be repeated on it with the pump running at the moment of the cut, so that the replayed history shows the device stopping the pump at 70 percent on its own.

8.6 Reducing transport latency at the radio

E9. The spread of the first round trip figures was wider than a local wireless network should produce, with a 95th percentile near 900 milliseconds at a signal strength better than -20 dBm. The cause was the radio: the Arduino port for the ESP32 enables modem sleep by default, so the receiver is powered down between beacon intervals and a packet waits for the next scheduled wake. Disabling it is a single call after association. The experiment held everything else fixed.

Table 8.6.1: Command round trip before and after disabling Wi-Fi power saving.

Statistic	Power saving on	Power saving off	Change
Minimum (ms)	144	29	-80%
Median (ms)	462	187	-60%
95th percentile (ms)	884	315	-64%
Maximum (ms)	906	402	-56%
Samples	12	12	

The improvement is largest in the tail, the signature of a scheduling effect rather than a bandwidth effect. The cost is roughly double the radio's continuous current, immaterial for a mains powered controller and decisive for a battery powered one; a latency figure quoted without its power cost is not an engineering result. The change does not alter the 200 millisecond control period, and therefore does not change how quickly the tank reacts to its own level.

8.7 Limitations

The group believes that pointing precisely at where the design falls short carries as much engineering value as presenting what worked.

On hardware, the most serious limitation is the flow measurement. The assignment makes a flow sensor mandatory, and two are fitted, but in this installation they do not measure water: the pump's conducted interference is forty times the real pulse rate, and the outlet draw is below the sensor's starting flow. Flow is therefore derived from the level, which makes the inlet a calibrated constant rather than an independent observation. Three consequences follow and are stated rather than hidden: the volume error of -5 percent with a 23 point spread is a property of that constant; the literal rule "pump on with near zero flow" is carried by level progress instead of the turbine; and the edge leak rule is inactive. The remedies are known and cheap: power the sensors from a supply that does not share a return path with the motor, filter the interrupt input by minimum pulse width (a real pulse at 0.36 litres per minute lasts about 14 milliseconds, the interference well under one), and choose a sensor whose range starts near 0.05 litres per minute.

Second, the current scale factor assumes the 5 ampere sensor variant and a one to one divider; it yields 360 to 425 milliamperes for a pump rated at 100 to 200, so either the divider or the variant differs from the assumption, and one reading with a multimeter in series will settle it. The binary use of current, present or absent, which is all the fault rules and the health classifier need, is unaffected. Third, the beam of the ultrasonic sensor is wider than the tank, so a share of pings return from the wall or the floor; the filter chain of Section 6.2 makes this manageable, and the counters show 93 percent acceptance, but a stilling tube or a wider vessel would remove the cause. Fourth, the sensor can lock up after a supply disturbance and recover only when its supply is cycled; powering it from a controllable output would let the firmware do that by itself.

On measurement, the headline latency no longer relies on two clocks, as Section 8.5 explains; the residual limitation is that the clock synchronisation error was characterised only by observation. The outage buffer protects only samples generated after the client notices the broken connection, so up to one keep alive interval, 15 seconds, is lost at the start of every outage; and an outage that also restarts the broker discards the replayed batch, since the device can flush before the backend resubscribes. Repeat counts of six to seventy five suffice for means and spreads but not for formal hypothesis tests. The rolling baseline needs at least three observations of each half hour slot before it may alarm; after three bench days none of the 48 slots qualifies, so its detection performance is unmeasured. The level calibration and three of the fault delays of Table 8.4.1 remain to be run on the final installation.

On operations, the public tunnel address changes every time it is started, which suits a demonstration but not a deployment, where a named tunnel or a fixed host is needed; administrator sessions are held in memory and do not survive a restart; and there is one administrator password rather than per user accounts. The broker traffic is not encrypted inside the laboratory network. There is no remote firmware update.

On the interface, a defect that survived three separate inspections is worth recording because of how

it hid rather than because of its size. The query that feeds the history charts applied its row limit after ordering the window in ascending time, so it returned the oldest rows of the window rather than the newest. At the specified rate of one message per second a ten minute window holds six hundred rows and never reaches the two thousand row limit, so the fault was invisible in every ordinary session. Running the simulator at ten times real time raises the rate to roughly five messages per second, the window then holds close to three thousand rows, and the charts silently froze in the past while the live status panel beside them continued to update. Nothing failed and nothing was logged; the two halves of the same screen simply disagreed.

The general lesson is that a pagination limit at the query layer must truncate from the end that is being discarded, not from the end being displayed. Ordering ascending and then limiting is correct only while the result set is known to fit, which makes it a latent fault that waits for the data rate to rise. The corrected query selects the newest rows first and reorders them for plotting. The group notes that the defect was found only because the acceleration factor of the simulator pushed the system past a threshold the specified rate never approaches, which is an argument for exercising a system deliberately outside its nominal operating point rather than only at it.

Future directions include a working flow measurement by the remedies above, which would restore the inlet as an independent observation and let the turbine and level estimates be checked against each other as a leak test in their own right; a Kalman filter in place of the alpha-beta filter, using the pump state as a known input; continuous speed control by pulse width modulation; and multiple nodes feeding one backend.

8.8 Faults found on the bench

The faults recorded here were all found after the prototype was assembled and nominally working. The group presents them because each was invisible from the code alone and each was resolved by a measurement rather than by argument.

8.8.1 An inverted relay made the firmware believe the pump was off

The most serious fault was also the simplest. The configuration declared the relay module active low, so the firmware drove the control pin high to stop the pump. The module fitted was active high. Every command to stop therefore started the pump, and the telemetry reported `pump: false` throughout. The tank filled from 0.2 to 11.2 centimetres and came close to overflowing while the interface displayed a stopped pump and an overflow lock.

The fault also corrupted the current measurement, in a way that concealed it. The quiescent offset of the Hall sensor is tracked by an exponential average that updates only while the firmware believes the pump is off. Because the firmware believed that permanently, the average absorbed the running current of the pump as its own zero, and the reported current settled near zero while the motor ran.

Determining the true polarity required care, because the obvious method gives the wrong answer.

The Hall current sensor is bidirectional: its output rests at mid scale and moves up or down depending on the direction of current through the primary conductor. A larger reading therefore does not imply a larger current, and the first version of the group's test concluded the opposite of the truth on exactly that basis. The quantity that does separate the two states is the *ripple*, because a brushed motor draws a visibly rough current through its commutator while an open relay draws a flat one. Measured over two rounds, the control pin low gave a mean of 1545.88 millivolts with 4.47 millivolts of ripple, and the pin high gave 1516.54 millivolts with 27.05 millivolts of ripple. The ripple ratio of 6.0 identifies the energised state unambiguously and does not depend on which way the load is wired, whereas the means differ by only 29 millivolts and in the misleading direction.

8.8.2 An automatic direction level shifter consumed a third of the echoes

The echo line of the ultrasonic sensor was initially routed through a TXS0108E translator. That part is intended for bidirectional open drain buses: it carries integrated pull up resistors of about 10 kilohms on both ports and a one shot circuit that accelerates edges. The echo output of an HC-SR04 is a strong push pull driver, and a pulse width is the only carrier of information in the signal, so both mechanisms work against it. Replacing the translator with a two resistor divider on the echo line alone changed the miss rate from between 28 and 30 percent of pings to zero in 179 consecutive samples.

8.8.3 Fifty hertz on a floating signal line reads as plausible flow

Both flow inputs reported between 0.26 and 0.51 litres per minute with the pump stopped. Converting the reading back to its underlying pulse rate showed exactly 50.00 hertz on one channel, unchanging across every sample. The control experiment was to close the drain valve, after which no flow of any kind was physically possible; the channel continued to report 50.00 hertz. A signal locked to the mains frequency and indifferent to whether water can flow is induced interference, not measurement.

The cause is impedance. The internal pull up of the microcontroller is about 45 kilohms, which leaves the line high enough in impedance to act as an antenna. The specified remedy, an external 4.7 kilohm pull up to the 3.3 volt rail, lowers the source impedance by an order of magnitude. A second and larger disturbance appears only while the pump runs, when both channels rise together to about 4750 hertz, a figure that would correspond to 48 litres per minute against a measured pump delivery of 0.36. Two independent sensors agreeing to within 0.7 percent are not measuring water; they are sharing a common mode disturbance injected by the motor.

8.8.4 A silently truncated message stopped the database without stopping anything else

Adding two validity fields to the telemetry payload pushed its length past the 448 byte buffer that served it. The JSON library truncates without raising an error and still returns a string, so the device published well formed nonsense. On the server the parse exception was raised inside a callback whose

exceptions the client library discards. The result was a system in which the device published once per second, the broker delivered every message, the health endpoint reported the service as connected, and the database received nothing at all for several minutes while the dashboard displayed a frozen value.

The lesson the group takes is that a serialiser which truncates silently and a callback which swallows exceptions are individually defensible and jointly produce a failure with no symptom at the point of failure. Both ends now report the condition explicitly, and the buffer carries margin above the maximum payload rather than matching it.

8.8.5 A command that worked but was never confirmed

Manual pump commands were executed by the device, yet the command log kept them as pending indefinitely, while mode commands were confirmed normally. The broker log showed the difference: acknowledgements of mode commands were 118 bytes long, acknowledgements of pump commands exactly 256 bytes, the size of the acknowledgement buffer.

The cause was an interaction between two libraries. The MQTT client delivers an incoming message as a pointer into its internal buffer, and the same buffer is reused for outgoing messages. The JSON parser, handed a non-constant pointer, parses in place and returns string references into that buffer rather than copies. A pump command starts the pump, which publishes the new pump state before the acknowledgement is sent; that publish overwrote the buffer, so the command identifier being echoed back had already become arbitrary bytes. Mode commands publish nothing in between and were unaffected. The fix copies the incoming message into a buffer owned by the handler before parsing. The general point is that a zero-copy parser makes the lifetime of the input part of the program's correctness, and that lifetime is decided by another library.

8.8.6 A robust filter gated by a fragile statistic

After the level filter was widened to a fifteen sample median, the group counted, for every ping, which validation stage rejected it. In one ordinary fill of 605 pings, 244 were rejected, and 98 percent of those rejections came from a single stage: the check that the spread of the window stayed below a limit. Missing echoes accounted for only 2 percent. The sensor was answering almost every time.

The spread had been measured as maximum minus minimum. That statistic is controlled entirely by the single most extreme sample, so one stray echo from the tank floor among fifteen good ones was enough to discard the whole window. The median was chosen precisely because it tolerates isolated outliers; gating it with the statistic most sensitive to isolated outliers undid that choice. Replacing the range with the interquartile range, which ignores the top and bottom quarter of the window, raised the acceptance rate from 60 to 93 percent in otherwise identical conditions. The interquartile range still grows when the sensor genuinely alternates between the water surface and the floor, because then roughly half the samples sit in each group, which is exactly the case the check exists to reject.

8.8.7 The level sensor is excellent, and unusable, in the same installation

With the water still, twenty consecutive samples read between 3.87 and 3.89 centimetres, a standard deviation of 0.010 centimetres. With the pump running, the same sensor collapses to reporting an empty tank while the tank is demonstrably filling. The group verified that the validity gates were not the cause by disabling them: the raw reading itself falls to the empty value, so the gates were reporting the data correctly rather than rejecting it wrongly.

This pair of results is the clearest argument in the project for layered protection. Throughout the interval in which the level reading was false and the validity flag had been forced true, the rule that stops the pump when the level fails to rise still fired and still stopped it. That rule consults the clock and the level history, never the validity flag, which is precisely why it survived.

8.9 Conclusion

The project delivers a working closed loop water tank that measures its level, controls its pump, detects and classifies faults, keeps working without the network, and can be watched from anywhere on the Internet while only an administrator can operate it. Every line of firmware, backend and interface was written by the group, and every figure in this chapter can be regenerated from the database by the scripts in the repository.

Three decisions carry the design. First, the whole control law and every safety rule live in the edge controller, derived from the rule that anything needing an immediate safe stop belongs at the edge; the outage requirement then follows from the architecture instead of being added to it. Second, safety is structural: every path that can start the pump passes through one guard function that is re-evaluated every cycle, and overflow is guarded by independent layers of which the raw distance guard, the float and the run limits do not depend on the filtered level at all. Third, the group trusted measurements over datasheets and over its own expectations: the pump delivers a fifth of its rated flow, the flow sensors count interference rather than water, the level sensor needs a fifteen sample robust filter in this tank, and the analysis of the final session found an initialisation defect that started the pump after every restart. Each of these was found by counting, and each is fixed or stated.

On the bench the controller stopped every automatic fill between 70.1 and 70.8 percent with at most 1.85 percent of overshoot, switched 12.9 times per hour with no chatter, raised no false alarm in 84 minutes, estimated pumped volume to -5 percent on average, and answered commands with a median round trip of 42 milliseconds. The main open item is a trustworthy flow measurement, for which the cause and the remedy are both identified.

8.10 Member contributions

Table 8.10.1: Distribution of work among group members.

Member	Area of responsibility
[Full name of member 1], [ID]	[Fill in area of responsibility]
[Full name of member 2], [ID]	[Fill in area of responsibility]
[Full name of member 3], [ID]	[Fill in area of responsibility]
[Full name of member 4], [ID]	[Fill in area of responsibility]
[Full name of member 5], [ID]	[Fill in area of responsibility]

Suggested grouping for the five members: hardware and circuitry; firmware and control; experiments and analysis; backend, database and security; interface and report. The assignment requires every member to present and answer technical questions, so the table records who led each area rather than limiting anyone's scope of understanding.

Deliverables are the working physical prototype; the firmware with its per-component test programs; the backend, dashboard and simulator; the protocol specification; the test procedures and analysis scripts (`docs/TESTING.md`, `tools/`); a code guide for newcomers (`docs/CODE_GUIDE.md`); the raw database; the presentation slides; and a backup demonstration video.

Bibliography

- [1] H. T. Thien, “IoT foundations and hardware architectures,” Lecture notes, IoT Foundations and Applications, Week 1, Ho Chi Minh City University of Technology and Education, 2026.
- [2] ITU-T, “Recommendation Y.2060: Overview of the Internet of Things,” International Telecommunication Union, 2012.
- [3] C. E. Shannon, “Communication in the presence of noise,” *Proceedings of the IRE*, vol. 37, no. 1, 1949, pages 10 to 21.
- [4] Espressif Systems, “ESP32 series datasheet,” Technical documentation, 2024. [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>
- [5] H. T. Thien, “IoT application layer messaging,” Lecture notes, IoT Foundations and Applications, Week 3, Ho Chi Minh City University of Technology and Education, 2026.
- [6] OASIS, “MQTT version 3.1.1,” OASIS Standard, 2014, published 29 October 2014.
- [7] E. Rescorla, “The transport layer security (TLS) protocol version 1.3,” RFC 8446, IETF, 2018.
- [8] H. T. Thien, “IoT cloud platforms, big data and edge computing,” Lecture notes, IoT Foundations and Applications, Week 4, Ho Chi Minh City University of Technology and Education, 2026.
- [9] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
- [10] H. T. Thien, “RESTful web API design and JSON data serialization,” Lecture notes, IoT Foundations and Applications, Week 5, Ho Chi Minh City University of Technology and Education, 2026.
- [11] B. Blanchon, “ArduinoJson: JSON library for embedded C++,” 2024. [Online]. Available: <https://arduinojson.org/>
- [12] K. Ogata, *Modern Control Engineering*, 5th ed. Prentice Hall, 2010.
- [13] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008.
- [14] R. Isermann, *Fault-Diagnosis Systems: An Introduction from Fault Detection to Fault Tolerance*. Springer, 2006.
- [15] E. S. Page, “Continuous inspection schemes,” *Biometrika*, vol. 41, no. 1/2, 1954, pages 100 to 115.
- [16] F. M. White, *Fluid Mechanics*, 7th ed. McGraw-Hill, 2011.
- [17] H. T. Thien, “Connectivity and network protocols,” Lecture notes, IoT Foundations and Applications, Week 2, Ho Chi Minh City University of Technology and Education, 2026.

- [18] H. T. Thien, “Frontend development, mobile apps and edge AI,” Lecture notes, IoT Foundations and Applications, Week 6, Ho Chi Minh City University of Technology and Education, 2026.
- [19] SQLite Consortium, “SQLite documentation,” 2024. [Online]. Available: <https://www.sqlite.org/docs.html>
- [20] I. Fette and A. Melnikov, “The WebSocket protocol,” RFC 6455, IETF, 2011.
- [21] Eclipse Foundation, “Eclipse Mosquitto: An open source MQTT broker,” 2024. [Online]. Available: <https://mosquitto.org/>
- [22] J. P. Bentley, *Principles of Measurement Systems*, 4th ed. Pearson Prentice Hall, 2005.
- [23] JCGM, “Evaluation of measurement data: Guide to the expression of uncertainty in measurement,” JCGM 100:2008, 2008.